

ORDBs

Data models in database systems

Lecture 3

Mara Romanovska

Mara.Romanovska@rtu.lv

25.09.2025.



RTU

**DATORZINĀTNES UN
INFORMĀCIJAS
TEHNOLOĢIJAS FAKULTĀTE**

Demo



Relationships



Logical types of relationships

In RDBMSs

- *One-to many*
- *Many-to-many*
- *One-to-one*

In OO programming

- Inheritance
- Composition (aggregation)
- **Association**

Association links

- «Normal» relational database approach vs. pure object approach
- One-to-one/one-to-many: ***object references***
- One-to-many/many-to-many: ***object collections***

Composition (aggregation links)

- Implemented by using collections and constraints
- NOT NULL
- CHECK

Object references



Reference type

- Object identifiers (OIDs) uniquely identify row objects in object tables
- Allow accessing other objects
- A REF is a logical pointer or reference to a row object that you can construct from an object identifier (OID)
- Used to **obtain, examine, or update** the object
- Reduces the need to build foreign keys

Defining object references

- Table person can contain REF to address instead of full address

```
CREATE TYPE ADDRESS_T AS OBJECT (  
    STREET    VARCHAR2(20),  
    NUMBER    NUMBER,  
    CITY      VARCHAR2(20));
```

```
CREATE TYPE PERSON_T AS OBJECT (  
    ID          NUMBER,  
    SURNAME     VARCHAR2(15),  
    NAME        VARCHAR2(15),  
    ADDRESS REF ADDRESS T );
```

```
CREATE TABLE ADDRESSES_TABLE OF ADDRESS_T;
```

```
CREATE TABLE PERSONS OF PERSON_T  
(ADDRESS with ROWID scope IS ADDRESSE_TABLE);
```

Inserting data into tables with REF

- Firstly we insert one record into each table, but the REF is left to be NULL

```
INSERT INTO ADDRESSES_TABLE VALUES (ADDRESS_T('Stabu', 70,  
    'Riga'));
```

```
INSERT INTO PEOPLE VALUES (1, 'Koks', 'Juris', NULL);
```

Creating REF

```
DECLARE
    adr_ref REF ADDRESS_T;
BEGIN
    SELECT REF(A) INTO adr_ref
    FROM ADDRESSES_TABLE A
    WHERE A.A_NUMBER = 70;

    UPDATE PEOPLE A
    SET A.ADDRESS = adr_ref
    WHERE A.P_ID =1;
END;
```

Notation of REF

```
SELECT * FROM PEOPLE;
```

ID	SURNAME	NAME	ADDRESS
----	---------	------	---------

1	Koks	Juris	
---	------	-------	--

<u>00002202088F3BF27A318C48D990A9AFB8668BE625DEC8EF0686AF4BCF8</u> <u>2580DD8AB6806E</u>			
---	--	--	--

DEREF operator

- REFs can not be read by humans
- REFs are used to access corresponding objects
- Such an access to an object through the REF is named dereferencing
- Oracle offers Deref operator to get object if the REF is known

Using Deref operator. Example

```
SELECT Deref(A.ADDRESS) FROM PEOPLE A;
```

```
Deref(A.ADDRESS) (STREET, NUMBER, CITY)
```

```
-----
```

```
ADDRESS_T('Stabu', 70, 'Rīga')
```

Dangling references

- Dangling objects are the ones that point to row objects with no object
- IS DANGLING

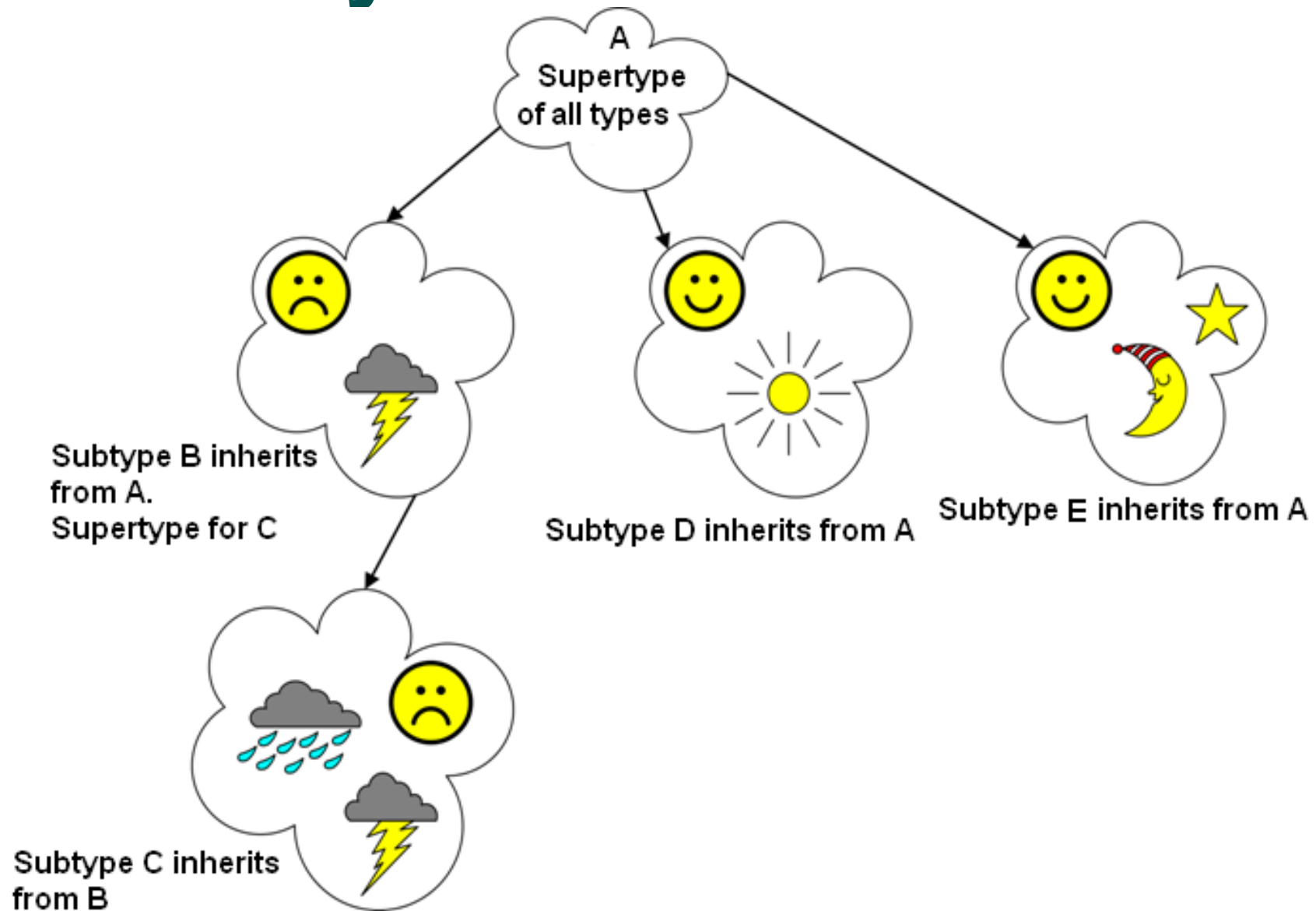
Inheritance



Type inheritance in ORDB

- ORDB allows to specialize object types by defining subtype(-s)
- Subtypes are more specialized versions of their parents
- Subtype has all its parent's attributes and methods
- Subtype has some additional properties that are defined in the subtype
 - Some additional attributes
 - Some additional methods
- Subtypes are created by deriving them from the parent object type that is said to be **supertype** of the subtype
- Subtypes are related to supertypes by the relation of inheritance
- Subtypes and supertypes that are connected with inheritance form a hierarchy of types

Type hierarchy



Polymorphism

- Instance of the supertype in the main code can be substituted with instance of any subtype. It is called **polymorphism**
- **Polymorphism** is defined as existence of attributes or methods with the same name that have different functions. It allows to standardize names of functions that do similar (but not exactly the same) things but have different implementations in different subtypes

Creating subtypes

- During the definition of subtype the supertype can be modified in the following ways:
 - Add new attributes
 - Add new methods
 - Change the actions done by some method

Forbidding the inheritance

- The type definition allows to specify whether it is possible to create subtypes of the type
 - Keywords FINAL (default) and NOT FINAL
- Methods also can be defined as FINAL

Creating subtypes

- Keyword Under

```
CREATE TYPE Student UNDER Person_T (  
    student_ID          varchar2(10),  
    course              number,  
    s_group             varchar2(5));
```

Union inheritance (1)

```
CREATE OR REPLACE TYPE Person_T AS OBJECT (  
  id VARCHAR2(10),  
  name VARCHAR2(20),  
  address VARCHAR2(35)) NOT FINAL;
```

```
CREATE TABLE Person OF Person_T (id NOT NULL, PRIMARY KEY (id));
```

```
CREATE OR REPLACE TYPE Student_T UNDER Person_T (  
  course VARCHAR2(10),  
  year VARCHAR2(4));
```

Union inheritance (2)

```
CREATE TABLE Student OF Student_T (id NOT NULL, PRIMARY KEY (id));
```

```
CREATE OR REPLACE TYPE Staff_T UNDER Person_T (  
department VARCHAR2(10),  
room_no VARCHAR2(4));
```

```
CREATE TABLE Staff OF Staff_T (id NOT NULL, PRIMARY KEY (id));
```

Mutual exclusion inheritance

```
CREATE OR REPLACE TYPE Employee_T AS OBJECT (  
  id VARCHAR2(10),  
  name VARCHAR2(20),  
  address VARCHAR2(35),  
  emp_type VARCHAR2(8)) NOT FINAL;
```

```
CREATE TABLE Employee OF Employee_T (  
  id NOT NULL,  
  emp_type CHECK (emp_type in ('Manager', 'Worker', 'NULL')),  
  PRIMARY KEY (id));
```

```
CREATE OR REPLACE TYPE Manager_T UNDER Employee_T (annual_salary NUMBER);
```

```
CREATE OR REPLACE TYPE Worker_T UNDER Employee_T (weekly_wage NUMBER);
```

Partition inheritance

```
CREATE OR REPLACE TYPE Employee_T AS OBJECT (  
id VARCHAR2(10),  
name VARCHAR2(20),  
address VARCHAR2(35),  
emp_type VARCHAR2(8)) NOT FINAL;
```

```
CREATE TABLE Employee OF Employee_T (  
id NOT NULL,  
emp_type NOT NULL CHECK (emp_type in ('Manager', 'Worker', 'Casual')),  
PRIMARY KEY (id));
```

```
CREATE TYPE Manager_T same as in mutual exclusion inheritance
```

```
CREATE TYPE Worker_T same as in mutual exclusion inheritance
```

```
CREATE OR REPLACE TYPE Casual_T UNDER Employee_T (hourly_rate NUMBER);
```

Object views



Object views

- Till now we analysed how to store data as objects
 - It fits only the situation when a new database is created
- Oracle allows to create objects on the bases of some relational tables
- *Object views* allow to create an object oriented abstraction based on the relational data
 - It is a solution that allows to create new object oriented application without modifying the schema of relational database

Characteristics of an object view

Object view has similar characteristics with the object table

- Object view contains object rows, fields match object's higher level attributes
- Each object has an identifier
 - Identifier is not generated by the system, but is user specified
 - Usually primary key of the base table is used as an identifier
 - It is defined by the WITH OBJECT ID clause in the CREATE VIEW command
- Object views are renewable

Creating object views. Example

Creating relational table

```
CREATE TABLE Employees (  
    E_ID          number (5),  
    SURNAME       varchar2 (20),  
    SALARY        number (9, 2),  
    POSITION       varchar2 (20) );
```

```
INSERT INTO EMPLOYEES VALUES (1, 'Koks', 200.50, 'carpenter');  
INSERT INTO EMPLOYEES VALUES (2, 'Zars', 180.00, 'secretary');
```

Creating object view

- Create an object type that will have exactly the same attributes as the previously created table

```
CREATE OR REPLACE TYPE Employee_Type AS OBJECT(  
    E_ID          number (5),  
    SURNAME       varchar2 (20),  
    SALARY        number (9, 2),  
    POSITION       varchar2 (20) );
```

- Create an object view where each row is an object of type Employee_Type:

```
CREATE VIEW EMP_VIEW OF Employee Type WITH OBJECT ID (E_ID) AS  
    SELECT A.E_ID, A.SURNAME, A.SALARY, A.POSITION  
    FROM Employees A;
```

- After creating an object view it can be used identically as an object table

Creating Object View Hierarchy

Three possible storage models:

- **a flat model**, all views in the object view hierarchy are based on the same table;
- **a horizontal model**, each view has a one-to-one correspondence with a different table;
- **vertical model**, the views are constructed using joins.

Flat model

```
CREATE TABLE AllPersons  
( typeid NUMBER(1),  
  ssn NUMBER,  
  name VARCHAR2(30),  
  address VARCHAR2(100),  
  deptid NUMBER,  
  major VARCHAR2(30),  
  empid NUMBER,  
  mgr VARCHAR2(30));
```

```
CREATE VIEW Person_v OF person_typ  
  WITH OBJECT OID(ssn) AS  
  SELECT ssn, name, address  
  FROM AllPersons  
  WHERE typeid = 1;
```

```
CREATE VIEW Student_v OF student_typ UNDER Person_v  
  AS  
  SELECT ssn, name, address, deptid, major  
  FROM AllPersons  
  WHERE typeid = 2;
```

Horizontal model

```
CREATE TABLE only_persons  
( ssn NUMBER,  
  name VARCHAR2(30),  
  address VARCHAR2(100));
```

```
CREATE TABLE only_students  
( ssn NUMBER,  
  name VARCHAR2(30),  
  address VARCHAR2(100),  
  deptid NUMBER,  
  major VARCHAR2(30));
```

```
CREATE OR REPLACE VIEW Person_v OF person_typ  
WITH OBJECT OID(ssn) AS  
SELECT *  
FROM only_persons;
```

```
CREATE OR REPLACE VIEW Student_v OF student_typ  
UNDER Person_v  
AS  
SELECT *  
FROM only_students;
```

Vertical Model

```
CREATE TABLE all_personattrs  
( typeid NUMBER,  
  ssn NUMBER,  
  name VARCHAR2(30),  
  address VARCHAR2(100));
```

```
CREATE TABLE all_studentattrs  
( ssn NUMBER,  
  deptid NUMBER,  
  major VARCHAR2(30));
```

```
CREATE OR REPLACE VIEW Person_v OF person_typ  
WITH OBJECT OID(ssn) AS  
  SELECT ssn, name, address  
  FROM all_personattrs  
  WHERE typeid = 1;
```

```
CREATE OR REPLACE VIEW Student_v OF student_typ  
UNDER Person_v  
AS  
  SELECT x.ssn, x.name, x.address, y.deptid, y.major  
  FROM all_personattrs x, all_studentattrs y  
  WHERE x.typeid = 2 AND x.ssn = y.ssn;
```

Q&A

